1. Introdução

A utilização de sistemas de informação é fundamental para a organização e o gerenciamento das atividades realizadas em uma clínica veterinária, que precisa lidar diariamente com diferentes tipos de informações, como dados de clientes, animais, produtos, serviços, pedidos e registros administrativos. Nesse contexto, a utilização de um banco de dados estruturado permite centralizar essas informações e contribuir para sua organização, integridade e disponibilidade.

Este projeto tem como objetivo desenvolver e organizar um banco de dados para a clínica veterinária fictícia Melhor Amigo. O sistema foi pensado para armazenar e organizar informações importantes para o funcionamento da clínica, como dados de clientes, animais de estimação, produtos, serviços, pedidos, estoque, funcionários e informações relacionadas à área contábil.

Inicialmente, o banco de dados havia sido desenvolvido utilizando MySQL. Com a evolução do projeto, surgiu a necessidade de realizar a migração para o PostgreSQL, adequando a estrutura existente aos recursos e às características do novo SGBD adotado para a continuidade do projeto.

Durante essa reorganização, as tabelas foram divididas em três schemas: `site`, `adm` e `contabil`. Essa divisão foi feita para separar melhor as diferentes áreas do sistema e facilitar a organização e a manutenção do banco de dados.

Além da reorganização da estrutura, foram criadas views para facilitar consultas frequentes, constraints para contribuir com a integridade dos dados, regras para definir o comportamento dos relacionamentos em operações de alteração e exclusão e um trigger para automatizar a atualização dos saldos contábeis.

2. Metodologia

2.1 Migração do MySQL para PostgreSQL

O primeiro passo do trabalho foi adaptar a estrutura do banco de dados que já existia em MySQL para o PostgreSQL. Durante essa etapa, foi necessário fazer algumas alterações na sintaxe e na estrutura para que o banco pudesse funcionar corretamente no novo SGBD.

Também foram revisadas as tabelas, chaves primárias, chaves estrangeiras e outras restrições que já faziam parte do projeto. Depois dessas adaptações, os comandos SQL foram organizados em arquivos separados, de acordo com os schemas do banco.

2.2 Organização em schemas

Após a migração, o banco de dados foi dividido em três schemas: `site`, `adm` e `contabil`.

O schema `site` reúne as principais informações utilizadas pelo sistema da clínica, como clientes, pets, produtos, carrinhos, pedidos e serviços.

O schema `adm` reúne informações relacionadas à administração da clínica, como funcionários, estoque e histórico.

Já o schema `contabil` concentra as informações relacionadas à parte contábil, incluindo pagamentos, plano de contas e lançamentos.

Essa divisão contribui para a organização do banco de dados, uma vez que cada schema concentra tabelas relacionadas a uma determinada área do sistema, facilitando sua manutenção e gerenciamento.

2.3 Implementação de Views

Depois de organizar os schemas, foram criadas views com o objetivo de facilitar consultas realizadas com frequência.

No schema `site`, foi criada a view `clientes_pets`, que permite consultar os clientes juntamente com os seus respectivos pets. Dessa forma, a aplicação consegue obter essas informações sem precisar montar novamente o mesmo `JOIN`. Essa view é destinada à aplicação do site, permitindo que as informações dos clientes e seus respectivos pets sejam consultadas de forma simplificada.

Também foi criada a view `produto_catalogo`, que apresenta o nome e o valor dos produtos cadastrados. Essa view é utilizada pela aplicação para facilitar a consulta das informações apresentadas no catálogo de produtos aos usuários.

No schema `adm`, foi criada a view `lista_clientes`, que facilita a consulta dos dados dos clientes pela equipe administrativa. Essa view é destinada à equipe administrativa, permitindo consultar de forma simplificada os clientes cadastrados no sistema. Também foi criada a view `relatorio_funcionarios`, que reúne informações dos funcionários, como nome, setor, salário e benefício. Essa view é destinada aos responsáveis pela administração da clínica, facilitando a consulta de informações relacionadas aos funcionários.

A criação de views facilita o acesso às informações e evita a repetição de consultas que são utilizadas com frequência.

2.4 Implementação de Constraints

Para contribuir com a integridade dos dados, foram adicionadas constraints do tipo `CHECK`.

Na tabela `site.pet`, foi criada uma constraint para impedir que a idade de um animal seja cadastrada com um valor negativo. Outra constraint foi utilizada para impedir que o peso seja menor ou igual a zero.

Na tabela `site.pedido`, foi criada uma constraint para limitar os possíveis valores do campo de status. Dessa forma, o campo pode receber apenas os valores `pendente`, `pago`, `cancelado` ou `concluido`.

Também foram revisadas as chaves estrangeiras do banco de dados. Para os relacionamentos analisados, foram definidas regras de `ON DELETE` e `ON UPDATE`, utilizando `RESTRICT` e `CASCADE`. Para os relacionamentos em que a exclusão do registro pai não deve provocar a exclusão dos registros relacionados, foi

utilizado ON DELETE RESTRICT, preservando a integridade dos dados. Já nos relacionamentos em que a alteração da chave deve ser propagada para os registros dependentes, foi utilizado ON UPDATE CASCADE.

Essas regras ajudam a controlar o que acontece com os registros relacionados quando uma informação é alterada ou excluída, evitando problemas de integridade entre as tabelas.

2.5 Trigger para atualização automática dos saldos contábeis

Foi desenvolvido um trigger para automatizar a atualização dos saldos das contas contábeis após a inserção de novos lançamentos na tabela `contabil.lancamentos`. O mecanismo utiliza a função `contabil.fn_atualiza_saldo()`, executada automaticamente após cada operação de inserção.

Na implementação realizada para este projeto, foi definida a seguinte regra: o valor informado no lançamento é acrescentado ao saldo da conta indicada como débito e subtraído do saldo da conta indicada como crédito. A função também verifica se os campos de débito, crédito e valor estão preenchidos antes de realizar as atualizações.

Para validar o funcionamento, foi realizado um teste com um lançamento de R$100,00 entre as contas Caixa e Fornecedores a Pagar. Após a inserção, o saldo de Caixa passou de R$5.000,00 para R$5.100,00, enquanto o saldo de Fornecedores a Pagar passou de R$4.000,00 para R$3.900,00, confirmando o comportamento esperado.

Também foram analisados seis lançamentos registrados em abril de 2026, considerando os saldos iniciais zerados para essa validação. A consulta agrupada por natureza das contas apresentou um saldo total de R$39.000,00 para as contas de natureza devedora e −R$39.000,00 para as contas de natureza credora. Os resultados confirmaram o equilíbrio entre os lançamentos e o comportamento esperado da regra implementada.

Como limitação da implementação atual, o trigger atua somente após a inserção de registros (`INSERT`). Alterações (`UPDATE`) ou exclusões (`DELETE`) de lançamentos não provocam a atualização automática dos saldos, sendo necessário implementar mecanismos adicionais para contemplar essas operações.

2.6 Organização do projeto

Além da organização do banco de dados, os arquivos do projeto também foram reorganizados para facilitar sua utilização.

Os scripts SQL foram separados de acordo com os schemas e armazenados na pasta `scripts`. O arquivo `site.sql` contém as estruturas relacionadas ao schema `site`, enquanto `adm.sql` contém as estruturas do schema `adm` e `contabil.sql` contém as estruturas do schema `contabil`.

Também foi criada a pasta `docs`, onde foram armazenados os modelos lógicos relacionais desenvolvidos no brModelo e demais documentos utilizados na documentação do projeto.

Por fim, foi criado um arquivo `README.md` com uma explicação sobre o projeto, sua organização e a ordem correta para executar os scripts. A sequência definida é `site.sql`, `adm.sql` e `contabil.sql`, pois existem dependências entre algumas tabelas dos diferentes schemas.